TECHNICAL SHEET

# Oktavia
# Multi-vendor network automation

Oktavia discovers the network, checks it against the customer standard, proposes the change against declared intent and, once a person approves, applies it and confirms the finding closed. One cycle, four vendors in production, every action with a downloadable artifact.

Built by Oktanet | Mexico · 2026 | Public document

## CONTENTS


1. Executive summary
2. The problem it solves
3. Value proposition
4. Capability categories
5. Platform modules
6. The closed loop
7. New service rollout
8. Site and device lifecycle
9. Argos, the network ChatOps
10. Local AI and documentation
11. Security and service inventory
12. Per-vendor coverage
13. Integrations
14. Architecture and operating model
15. Platform security
16. Integration requirements
17. Implementation process
18. Licensing
19. Use cases


## 1. Executive summary

Oktavia is a network automation platform for enterprise IT operations. It pairs an automation engine with a unified web interface to run audits, verify configuration compliance, visualize network state, and manage changes in a controlled and traceable way.

It is built for teams that need to move fast without giving up control. Every action produces versioned artifacts, every change requires prior validation, and everything lands in a searchable history.

> **What sets Oktavia apart:** true multi-vendor in production across Cisco IOS and IOS-XE, Fortinet FortiGate and FortiSwitch, and Juniper Junos, plus a service provider core on Cisco IOS XR over NETCONF; Argos as a ChatOps layer across the whole platform; local AI that answers from the customer documentation without anything leaving the server; network security with configuration evidence, a PCI DSS report per site, and security patches prepared for approval; service inventory; and the full detect, remediate, and verify cycle on all four vendors.

## 2. The problem it solves

Enterprise networks grow more complex faster than the teams that run them. The symptoms repeat across almost every customer:

- **Manual audits** that take days and depend on nobody misreading a configuration.
- **Configuration drift** that goes unnoticed until there is an incident.
- **Fragmented visibility** across inventory, compliance, topology, and metrics, each in its own tool.
- **No traceability** on who changed what, when, and with whose approval.
- **Reactive operations**: the team acts when a user reports, not when the network drifts.
- **Diagnosis without closure**: audit tools find the problem, but fixing it stays manual work outside the tool.

## 3. Value proposition

| BENEFIT | DESCRIPTION |
|---|---|
| **The full cycle, not just diagnosis** | Detect, propose, approve, apply, and verify the finding closed. Finding a problem is worthless if fixing it stays manual. |
| **Audits in minutes, not days** | Full compliance across the whole network, a site, a role, or a single device in one operation. |
| **Control without risk** | No change is ever applied without generated artifacts and prior validation. Each vendor is touched with its safest primitive. |
| **End-to-end visibility** | Inventory, topology, compliance, security, addressing, and configurations in a single console. |
| **Traceable automation** | Complete history of jobs, results, and downloadable artifacts in JSON, CSV, and CFG. |
| **Genuinely multi-vendor** | Each vendor has its own normalized configuration model, not a translation into Cisco syntax. |
| **Operating by conversation** | Argos audits, diagnoses, proposes configuration, and closes the loop from the chat, without switching screens. |
| **AI with its cost in plain sight** | Every answer states whether it consumed the language model or resolved deterministically, before it runs. |

## 4. Capability categories

Oktavia capabilities group into seven categories. It is the way to read the platform by problem solved, not by screen.

| # | CATEGORY | WHAT IT SOLVES |
|---|---|---|
| 1 | **Network state and configuration** | Knowing at all times what exists, how it is connected, and how it is configured. |
| 2 | **Compliance and configuration control** | Defining the standard and verifying the network meets it, with no blind changes. |
| 3 | **Remediation and service rollout** | Closing the finding and rolling out new services with approval, verification, and rollback. |
| 4 | **Network security** | Firewall and switch posture with the configuration evidence backing each finding. |
| 5 | **Observability and event automation** | Seeing operational health in real time and reacting automatically to network conditions. |
| 6 | **Intelligence and analysis** | Turning raw data into decisions: risk, drift, capacity, paths, and flows. |
| 7 | **Governance and FinOps** | Proving the return on automation and operating multi-tenant with traceability. |

## 5. Platform modules

| MODULE | FUNCTIONAL SCOPE | LICENSE |
|---|---|---|
| **Automation center** | Executive view of the network: global compliance score, devices reachable by ping and SSH, distribution by role, sites covered, active failures, and recent job activity. | Pro |
| **Domain consoles** | One console per network domain —Campus, WAN, Security and Management—, in the style of a cloud services console: summary, intent, rules, compliance, changes, Argos workflows and telemetry for the domain in one place, with the site as scope. Config intent and state intent read as a single table, value by value, and an administrator edits the config right there; picking a site leaves only what applies to its devices. Network-wide views remain available with their domain filter. | Pro |
| **Network inventory** | Dynamic multi-vendor inventory with filters by site, role, platform, vendor, and site type. Visual distribution, per-device detail, audit freshness, CSV import and export with non-destructive merge. | Pro |
| **Topology and path tracing** | Interactive map with physical, control-plane, routing, OSPF, IS-IS, BGP autonomous system, and STP views; in the core, point-to-point links and pseudowires. Per-node and per-site panels. Live path trace between two points with hop, next hop, egress interface and protocol, reverse path, and asymmetry detection, including across an MPLS L3VPN. | Pro |

| MODULE | FUNCTIONAL SCOPE | LICENSE |
|---|---|---|
| **Routing analysis** | A service that traces the paths from one site to the others: result, hops, protocols, encryption and symmetry, with the forward and return diagram and what was found on each device. Alongside it, what is running on the network by protocol —OSPF, BGP, IS-IS, EIGRP, FlexVPN and GRE tunnels, pseudowires, Spanning Tree, HSRP/VRRP and VLANs—, with its sessions, health and a link to its intent. | Pro |
| **Configuration compliance** | Global and per-site score, top failures, and trends. Per-device results with severity, diagnosis, and remediation plan. Editable service catalog and rules with selective enablement by role, vendor, platform, or site type. | Pro |
| **Desired configuration** | Configuration generated from service variables and templates, with precedence by defaults, site type, site, platform, model, role, and device. JSON and CSV export. | Pro |
| **Hardening standard by role** | The security standard lives in the device role and a new device inherits it when onboarded: syslog, SNMP, NTP, management access and services across the four vendors. On Cisco IOS, besides what is missing, it detects what is extra — HTTP server, public community, remote access without SSH— and the Digital Twin proposes removing it. | Pro |
| **Closed-loop remediation** | Audit, propose, review the diff, approve, apply, and audit again to confirm closure. The verdict compares each device's test results before and after the change: what was fixed, what broke, and what still fails; a regression prevents closing. Every step leaves a downloadable artifact and lands in the history. | Pro |
| **New service rollout** | The same cycle for rolling out services: tunnels between sites, advertising a network in OSPF and BGP, DHCP per VLAN, routing adjustments. Each with its rollback. | Pro |
| **Customer services** | For service providers: per customer, its L3VPNs and pseudowires with the live status of each one and its end-to-end diagram. | Pro |
| **Argos — network ChatOps** | Plain-language assistant embedded in every screen. It understands, investigates, configures, and closes the loop. Each action states whether it consumes the AI model before it runs. | Pro |
| **Local AI (RAG)** | Open models served on the Oktavia server itself that answer only from fragments retrieved from the documentation —official vendor guides, per-site technical report, the team's standards and procedures, previous change reports —, citing document, section, and page. Nothing leaves the server and there is no per-question cost. Includes the documentation kit with templates and validation. | Pro |
| **Network security** | Posture computed on collected configuration across four domains: perimeter, management plane, identities, and segmentation, plus vulnerabilities by software version (NVD, CISA KEV and FIRST EPSS). Global and per-device score, filters by domain, severity, site and device, evidence per finding, and what to address first. PCI DSS v4.0.1 network report per site and security patches: how to remediate each CVE, an urgent change in the Digital Twin, and an automatic flow that prepares the drafts without applying them. | Pro |

| MODULE | FUNCTIONAL SCOPE | LICENSE |
|---|---|---|
| **Service inventory** | Cross-check between what is observed on the device, what is declared as intent, and the service catalog. Classifies each service as covered, blind spot, not applied, or not applicable, with its configuration scope. | Pro |
| **Digital Twin** | Network state snapshots and comparison between desired and live configuration. Per-device drift detection, impact quantification, and an approval flow through successive states up to application. The «Security patches» category tracks each change until it is verified in the backup and fixed once the CVE disappears. | Pro |
| **Firewall policy analysis** | Which policy decides a flow at each hop along the path, with its address translation and its logging. Resolves object groups and application sets recursively. | Pro |
| **IP Intelligence (IPAM)** | Addressing map, detected subnets, overlaps, routing neighbors, and the location of an IP address down to the access port by crossing the ARP table with the MAC table. | Pro |
| **Configuration backups** | View, compare, and download configuration files by device and date. | Pro |
| **Advanced reports** | Network health, configuration risk, drift, capacity, ports, and topology, in downloadable reports ready for audit. | Pro |
| **Provisioning and site management** | Logical structure of site types, sites, and roles defined before the first discovery run. Executive per-site view with devices, management addresses, compliance, and network configuration. | Pro |
| **Operations and scheduled audits** | Running and scheduling audits, discovery, inventory sync, and connectivity validation, with job history and artifacts. | Pro |
| **FinOps** | Automation return-on-investment calculator and operational savings analysis, with the estimated network cost per device, per site and per domain. | Pro |
| **Multi-tenant management** | Multi-organization, user management, and per-tenant data separation. | Pro |
| **Real-time telemetry** | Operational CPU, memory, interface and routing-neighbor metrics stored as time series, with temporal visualization, trends, and threshold alerts. Through SNMP polling on any vendor —on Cisco IOS XR also BGP sessions, IS-IS adjacencies and route counts— and, on Cisco IOS XE, through streaming: the device pushes its data every ten seconds without being polled. Enabled from Argos and applied through the Digital Twin. | Telemetry and Events |
| **Event automation** | Condition-to-action rules that react automatically to BGP session drops, OSPF neighbor drops, interfaces down, and CPU, memory, temperature, or error thresholds. | Telemetry and Events |

## 6. The closed loop

This is what separates Oktavia from an audit tool. Finding a problem is worthless if fixing it stays manual, and applying a change blind is worthless if nobody verifies it landed right. The loop is a single path walked without leaving the platform:

1. **Audit** the chosen scope: a device, a role, a site, or the whole network.
2. **Read the compliance result**: what failed, at what severity, and which configuration lines are missing.
3. **Propose the change** against declared intent, not against a generic template.
4. **Review the diff** between desired and live configuration in the Digital Twin, with estimated impact.
5. **Approve.** A person, always. No change applies on its own.
6. **Apply** using each vendor safe primitive.
7. **Verify**: audit again and confirm the finding closed.

Every step leaves a downloadable artifact and lands in the job history. If the result is not convincing, there is a way back.

### Before and after verification

The verdict is not just «the rule passes now». Oktavia compares each device's test results before and after the change and splits them into **fixed**, **regressions**, **new failures**, and **still failing**. A regression prevents closing the loop. State failures that already existed before the change are shown separately and do not block it, because the change did not cause them.

Argos also closes the loop on changes that already exist in the Digital Twin, such as a security patch prepared by the automatic flow; a verified patch is marked as such. Every verdict is indexed as a change report in the local AI, so the next similar change starts from what was already learned.

### How it applies on each vendor

| VENDOR | APPLY PRIMITIVE | SAFETY NET |
|---|---|---|
| **Juniper Junos** | Deferred commit confirmation over NETCONF | The device rolls back on its own if nobody confirms within the window. A session that drops mid-apply does not leave the device unreachable. |
| **Fortinet FortiGate** | Operations over the configuration API | Changes touching administrators and zones stay in permanent simulation, because moving a zone invalidates the policies referencing it. |

| VENDOR | APPLY PRIMITIVE | SAFETY NET |
|---|---|---|
| **Cisco IOS XR** | NETCONF with prior validation (dry run) and commit confirmed | The device rolls back on its own if the commit is not confirmed. After the change, IS-IS, BGP and the pseudowires are verified before the change is accepted. |
| **Fortinet FortiSwitch · Cisco IOS** | Secure session with vault credentials | Diff before and verification after. The rollback is the inverse change, also reviewed and approved. |

## 7. New service rollout

The same cycle rolls out services, not just corrects deviations. You describe what you want and Oktavia proposes it as declared intent:

- **A tunnel between two sites**, with the routes that run through it, and its complete teardown when the project ends.
- **Advertising a network**: logical interface, OSPF announcement, access control lists, and BGP on the site edge devices, with ping verification.
- **Editing routing**: router identifier from the logical interface, recommended site summarization.
- **DHCP for a VLAN**, with the range proposed from what Oktavia already knows about the segment.
- **AAA** with TACACS+ or RADIUS and local fallback, or local only. The rollout is blocked if the device has no local fallback user, so nobody gets locked out, and the server key lives in the vault, not in the intent.
- **FlexVPN tunnel** (IKEv2 and IPsec), hub-and-spoke, with the pre-shared key in the vault, and the **migration from GRE** in three approved changes: BGP over the encrypted tunnel, removal of the old BGP session, and teardown of the GRE tunnel, including its intent and rules.
- **Device license**: the level is declared in the intent, and the audit verifies it became active after the reload, which the operator decides.
- **Port security**: which port was blocked, which MAC address caused it, and which one is allowed; the reset is an approved change, and the lab offers a guided demo.
- **Adopting what exists**: read a device backup and declare as intent what it already has configured, so auditing can start without redesigning anything.

> Every service is proposed, edited before applying, and ships with its rollback. Nothing is written to the network without a person approving it.

### Service provider core (Cisco IOS XR)

In the core, intent is declared per service from the catalog, and each service has its own desired configuration and its own compliance diff:

- **Hardening** of the device.
- **IS-IS with Segment Routing MPLS**, TI-LFA protection and BFD.
- **BGP VPNv4** with a route reflector.
- **L3VPN per customer**, with its VRF toward the CE.

- **L2VPN EVPN-VPWS pseudowires**, several per PE-CE link over VLAN subinterfaces.

Devices are managed model-driven over NETCONF, with the Cisco-IOS-XR-um configuration models and the native operational models, with no screen scraping. Every change goes through the Digital Twin: prior validation, commit confirmed and confirmation by a person.

Multi-step workflows show their progress per site —«X of N steps» and a percentage— computed from the real state of the network, not from boxes ticked by hand, and every card ends with the button for the next step.

## 8. Site and device lifecycle

Opening and closing sites, and onboarding, replacing and retiring devices, are one-button flows with preview, approval and verification. Each is first rehearsed on the test network (Cisco Modeling Labs), where Oktavia creates the nodes with a minimal day 0 and wires them; powering them on is always the operator's call.

- **Site creation**: the design of its site type is cloned, with proposed and editable addressing, the hub side (BGP, tunnel, subinterface, VLAN), and the lab nodes and test PCs.
- **Site removal**: cleans the hub in order (BGP `no neighbor`, tunnels, subinterfaces, VLANs and trunks), cleans Oktavia and the lab, and validates that nothing of the site is left. Create, remove and create again is repeatable.
- **Device onboarding**: access with the vault credential, discovery and audit, configuration from intent, telemetry, loop and verification.
- **Device replacement**: a different model under the same name, editable port mapping, link cutover, and the old device retired once the new one is powered on and in service (the practice is to verify it before retiring the old one). The new SSH key is accepted only with operator confirmation.
- **Device retirement**: a downloadable archive package without secrets, and cleanup of intent, inventory, compliance and telemetry.

On Cisco SD-WAN the same loop goes through the Manager: edge onboarding with its Config Group and bootstrap, and creation and removal of LAN segments. The details are in the article Test before you touch production: the Oktavia test network.

## 9. Argos, the network ChatOps

Argos is the assistant embedded in every Oktavia screen. You ask in plain language and it answers with network data, not generalities. It covers four moments of daily work:

- **Understand**: network state, compliance, what to address first, diagnosis of a device or a site.
- **Investigate**: path between two IP addresses forward and back, correlation of failures against configuration, anomaly and unreachable device detection.
- **Configure**: describe the service in plain language or paste the configuration, and Argos proposes it as declared intent for the operator to edit before applying.
- **Close the loop**: audit, read compliance, take it to the Digital Twin, approve, and verify. Each step is a button, not a sentence you have to type.

Workflows and domains follow each organization's architecture. In a service provider network, Argos offers the core, L3VPN and pseudowire workflows, and organizes the network into Transport and Services domains instead of Campus and WAN.

Two design principles separate it from a generic assistant. Every answer states whether it consumed the language model or resolved deterministically, so the cost is visible before you click. And no action that touches the network happens without a person approving it.

### What uses AI and what does not

Most of what Argos does never touches the model: they are queries and calculations over already collected data, with reproducible results. The model steps in when something has to be written, prioritized, or interpreted from free text.

| CAPABILITY | ENGINE |
|---|---|
| Network state, compliance, diagnosis by device or site | Deterministic |
| Service inventory and blind spots | Deterministic |
| Security posture and its findings with evidence | Deterministic |
| Path between two IP addresses, forward and back, with firewall policy per hop | Deterministic |
| Locating an IP address down to the access port | Deterministic |
| Importing a device configuration as declared intent | Exact parsers where they exist; AI for the rest |
| Closing the loop: audit, compliance, Digital Twin, verify | Deterministic |
| Starting from a ticket: recognizing the sites in the text, auditing their configuration and state, and summarizing the findings | Deterministic |
| Interpreting an already computed path: architecture and troubleshooting findings | AI |
| Explaining security: real priority and concrete per-vendor changes | AI |
| Recommending what to declare first from the service inventory | AI |
| BGP and OSPF routing analysis: state, risks, and recommendation | AI |
| Configuring by conversation: describing a service in plain language | AI |
| Correlating compliance failures with the configuration | AI only when there are findings |
| Optimizing scopes and proposing a standard | AI on top of a deterministic calculation |
| Progress of multi-step workflows per site | Deterministic |

| CAPABILITY | ENGINE |
|---|---|
| IOS XR core state: IS-IS adjacencies with level, area and adjacency SID; BGP with its ASN; routing table with VPN labels and TI-LFA backups told apart from ECMP; xconnect state | Deterministic |
| Path across an MPLS L3VPN: ingress PE, VPN label and egress PE, in both directions | Deterministic |
| Ports blocked by port security and their reset | Deterministic |
| Questions to the documentation and about how to do something in Oktavia, «What the documentation says» per incident, and «How to remediate» a CVE | Local AI, on the customer server |
| Preparing security patches: drafts in the Digital Twin and standard proposals, without applying | Deterministic |

The pattern repeats: the data is computed first, and the model only writes or prioritizes on that result. It never invents findings, it works with the ones the calculation produced.

Every action carries a visible badge before you click: ⚙ **IAOps** is deterministic, ◆ **IA local** (local AI) runs on the Oktavia server without sending anything or consuming tokens, and ✦ **IA** uses the cloud model with its per-query cost.

## 10. Local AI and documentation

Argos answers with network data; the local AI adds what the documentation says. It is a retrieval-augmented generation (RAG) system that runs entirely on the customer's Oktavia server: the models are open and served with Ollama —bge-m3 for embeddings and qwen2.5 7B for generation, by default— and the vectors are stored in the platform's own PostgreSQL database. Nothing leaves the server and there are no per-question tokens.

The business overview, with the flow diagram and an Argos video: Local AI with your company's context.

### How it answers

- **Only from what was retrieved.** The answer is built from the retrieved fragments and cites them: document, section and, for PDFs, page.
- **Verbatim commands only.** A configuration command is accepted only if it appears verbatim in a cited fragment; no syntax is invented.
- **Bounded scope.** The question is asked about a device, a site, or the whole network, and retrieval is limited to that scope.
- **Hybrid search.** Search by meaning (vectors) is combined with exact-word search (BM25), so it finds commands and names such as `snmp-server view` or FlexVPN.
- **From the document to the workflow.** When the cited source is an Argos workflow, the answer carries the button that opens it («Do it in Argos»). The buttons come from the product catalog, not from the model.
- **Saved answers.** The same question over the same sources is answered instantly.

- **Secrets are never indexed.** Lines mentioning communities, keys, or passwords are dropped, and the configuration standard is generated with secret fields masked.
- **Response time.** On CPU an answer takes roughly a minute; a GPU makes it much faster.

## What it indexes

| CORPUS | CONTENT |
|---|---|
| **Official vendor guides** | Selected by the platform families in the inventory, from verified sources that allow download: the Cisco IOS XE hardening guide from Cisco's security center, FortiOS 7.2 hardening and troubleshooting, Junos OSPF and BGP troubleshooting and security policies, Meraki MX troubleshooting, and the NSA network infrastructure security guide. Sites that block automated downloads (www.cisco.com, rfc-editor.org, nist.gov) are uploaded by an administrator. |
| **Uploaded documents** | Text PDFs, not scanned, up to 80 MB and indexed in the background; HTML and Markdown. Adding by URL only accepts an allow-list of vendor documentation domains, over https and never resolving to private addresses. |
| **Network technical report** | The technical report (LLD) for each site, re-indexed when asked about that site. |
| **Oktavia documentation** | The articles of the documentation portal, plus sheets generated from the product code: every Argos tab and button, and every Configurator service with its fields and an example without secrets. They are regenerated with the product, so they do not drift, and they are filtered by the vendors in the inventory. |
| **Change reports** | Every Argos closed-loop verdict, so it learns from previous changes. |

## Documentation kit

In Integrations → Local AI, a nine-item checklist shows what documentation the network has and what is missing. Part of it is generated by Oktavia: the configuration standard from the intent, what each compliance rule audits, and the technical report per site. The rest is written by the team with Markdown templates whose «(obligatoria)» (mandatory) sections are validated on upload: design standard (HLD), troubleshooting runbooks, change procedure, network security policy, and circuits and provider escalation. The ninth item is the vendor guides. The kit ships filled examples for three test networks: Cisco IOS hub-and-spoke, Cisco SD-WAN, and multi-vendor with FortiGate, Junos and Cisco.

## Where it is used

- Argos questions to the documentation («according to the documentation…», «argos local: …») and about how to do something in Oktavia («how do I migrate to FlexVPN in Oktavia?»), also from «Ask the documentation» in the Argos launcher.
- «What the documentation says» for each incident in the diagnosis.
- «How to remediate · Local AI» on each CVE: it reads the vendor advisory and extracts the fix and the mitigations.

## Measured quality

A set of reference questions, each with the source it must cite, measures whether retrieval brings back the right source. It runs on the server without using the answer model. On the Cisco IOS test network, adding

the documentation generated from the product took first-source hits from 4% to 93%, and to 100% within the six sources the model reads. Every wrong answer that is found is added to the set as a new question.

### Good practices

- **Only content you have the right to use.** Your own documentation, public vendor guides, and what the team is entitled to use. Copyrighted books and courses are not indexed.
- **Curate the sources.** Official, current guides for the versions running in the network; an old or third-party document produces old or foreign answers.
- **Text PDFs.** A scanned PDF has no text to retrieve; convert it before uploading.
- **Fill the kit's mandatory sections.** They answer the most questions and they are validated on upload.
- **Name runbooks after the symptom** («BGP neighbor down», «tunnel without traffic»), which is how people ask during an incident.
- **Regenerate Oktavia's part** —standard, rules, and technical report— after a major design change.
- **Size the server.** It works on CPU at roughly a minute per answer; for daily use, a GPU is worth it.
- **Escalate to the cloud only explicitly.** Sending a question to the cloud model is a visible action, marked ✦ IA, and only a summary bounded to the device or site travels, not the document base.
- **Do not upload secrets.** The system drops lines with credentials, but good practice is for documents not to carry them.

## 11. Security and service inventory

### Network security

Security posture computed on the collected configuration of firewalls and switches, across four domains: perimeter, management plane, identities, and segmentation. Each finding carries the configuration line backing it and its recommendation in the vendor own syntax. It includes a global and per-device score, and filters by domain and severity. On Cisco IOS it also checks access ports without port security or with a violation action that does not block, traffic between sites over unencrypted GRE tunnels, weak IKEv2 encryption, and reused pre-shared keys.

### Vulnerabilities by version

Each device's software version is matched against NVD, CISA KEV and FIRST EPSS: actively exploited vulnerabilities first, with a rating per device, site and tenant, the minimum safe version per model that feeds the software standard, and a printable CVE report per network or site. Lab images are marked as approximate and do not count as real exposure. More in the article Security in Oktavia and Argos.

### What to address first

A list of CVEs is not a decision. Argos crosses each exploited or likely vulnerability with the device's running configuration —whether the vulnerable feature is enabled— and with its role in the network, and classifies it as **act now**, **plan**, **watch** or **confirm version**. An edge device with the feature exposed goes first; an internal one with the feature off, later. The score runs from 0 to 100, higher is better, with bands, and CVEs can be filtered by site or device.

### PCI DSS network report per site

For the sites in scope of the cardholder data environment —the point-of-sale sites—, Oktavia builds the PCI DSS v4.0.1 network report on requirements 1, 2, 6.3.3, 7, 8, and 10. Each requirement is **met**, **not met**, or **not evaluated**, and it only counts if the device's vendor family has a check for it. It carries evidence per device and comes out as a printable PDF. It supports the QSA's work; it does not replace it.

### Remediation and security patches

- **How to remediate.** The local AI reads the vendor advisory and extracts the fixed version and the mitigation.
- **Urgent change.** It becomes a draft in the Digital Twin, in the «Security patches» category, with its board: proposed, approved, applied, verified when the lines appear in the backup, and fixed when the CVE disappears.
- **Declare in the Configurator.** When the mitigation line is a field of the standard —HTTP server off, SSH v2, VTY SSH only, SNMP views, password encryption, spanning-tree mode, logging trap—, it is written to the device's intent and compliance watches it from then on. Lines carrying a secret stay pending for the vault.
- **Security patches flow.** With a button, or by asking Argos to «prepare the security patches», it takes the exploited or critical CVEs whose feature is enabled, prepares the drafts and the standard proposals, and notifies. It never applies: approval stays human.

### Hardening standard by role

The security standard is declared once, in the role, and every device with that role inherits it, existing and new: logging to the collector, restricted read-only SNMP, NTP, management access over SSH only, and unnecessary services turned off. Other scopes keep only what varies per site or per device. Compliance checks what is missing and, on Cisco IOS, also what is extra; the Digital Twin proposes the fix and a person approves it.

### Service inventory

It answers the question that comes before compliance: what is configured on each device, and how much of it is declared as intent. It cross-checks what is observed against what is declared and against the service catalog, and classifies each service into four states, along with the configuration scope that defines it.

| STATE | MEANING |
|---|---|
| **Covered** | The service is configured on the device and declared as intent. Oktavia audits it. |
| **Blind spot** | It is configured on the device but nobody declared it. Nobody audits it, and a change to it goes unnoticed. |
| **Not applied** | It is declared as intent but does not appear in the device configuration. |
| **Not applicable** | It does not correspond to the platform or the role of that device. |

From the inventory itself, blind spots are declared as intent in a single step, without touching the network: what was already there simply starts being audited.

## 12. Per-vendor coverage

| VENDOR | ACCESS METHOD | COVERAGE |
|---|---|---|
| **Cisco IOS / IOS-XE** | SSH | Discovery, compliance, desired configuration, remediation |
| **Cisco IOS XR** | NETCONF (YANG) | Discovery, per-service compliance, desired configuration, remediation with commit confirmed, IS-IS, BGP and pseudowire state |
| **Fortinet FortiGate** | REST API | Discovery, compliance, desired configuration, remediation, firewall policies |
| **Fortinet FortiSwitch** | SSH | Discovery, compliance, desired configuration, remediation |
| **Juniper Junos** | NETCONF | Discovery, compliance, desired configuration, remediation, zone policies |

**Coverage grows with every project.** The architecture is multi-platform by design: each vendor has its own normalized configuration model, not a translation into Cisco syntax. Virtual domains and interfaces in FortiOS, contexts and zones in Junos. Integrating a new platform is four known pieces: discovery adapter, normalizer, compliance rules, and generation templates. It is bounded, repeatable work, done against the customer real installed base, in proofs of concept and projects.

## 13. Integrations

Oktavia was designed to coexist with the customer ecosystem, not to replace it. It can feed and be fed by the tools already in operation.

| INTEGRATION | STATUS | USE |
|---|---|---|
| **SNMP and metric collection** | Production | Native telemetry engine: polling and storage in a time-series database. |
| **Streaming telemetry (model-driven)** | Production | Cisco IOS XE pushes interfaces, BGP neighbors, CPU and memory to the Oktavia collector over gRPC every ten seconds. It feeds the same charts as SNMP polling, without duplicating series. |
| **MCP server** | Production | The engine exposes, read-only over the Model Context Protocol, the tenant inventory, compliance and artifacts, for Argos and for assistants such as Claude Desktop. |
| **Grafana** | Compatible | Oktavia time-series database is a direct Grafana datasource; the customer builds dashboards on the same metrics with no extra transformation. |

| INTEGRATION | STATUS | USE |
|---|---|---|
| **Zabbix and other SNMP systems** | Interoperable | Devices exposed over SNMP can be consumed by whichever monitoring system the customer already runs. |
| **Email (SMTP)** | Production | Notifications and alerts by email over any customer SMTP server. |
| **Anthropic (Claude API)** | Production | AI engine for analysis, recommendations, and conversational assistance. |
| **Ollama (local AI)** | Production | Open models on the Oktavia server for the local AI: bge-m3 for embeddings and qwen2.5 7B for generation, with the vectors in the platform's PostgreSQL database. |
| **NVD, CISA KEV, and FIRST EPSS** | Production | Public sources for vulnerabilities, active exploitation, and exploitation probability, for the software-version cross-check. |
| **HTTP webhooks** | Production | Event delivery to any HTTP integration point, which is the way to connect ticketing and messaging tools. |
| **External source of truth** | Optional | Inventory ingestion from an external system of record if the customer already runs one. |
| **CSV and REST API** | Production | Import and export of inventory, catalogs, and data across every module; programmatic integration over API. |

**Integration differentiator:** Oktavia contributes the intent and compliance layer, the standard the network is compared against, which a metrics dashboard or a traditional monitoring system does not have. They measure; Oktavia also validates against a standard and automates the response.

## 14. Architecture and operating model

The platform separates the web interface from the automation engine. The interface owns the user experience, tenant organization, and persistence; the engine executes against the devices. They talk over a REST API authenticated with a key.

- **Persistence:** relational database with per-tenant data separation.
- **Artifacts:** JSON, CSV, and configuration files, downloadable and versioned.
- **Deployment:** orchestrated containers, on customer premises or in a private cloud.
- **Device access:** SSH, NETCONF, and native vendor API, with credentials encrypted in a vault.

### Operating model

- Everything runs as **asynchronous jobs** with complete history.
- Every job produces **versioned artifacts**, downloadable.
- **No direct changes are ever applied** without generating and validating artifacts first.
- Remediation actions are **supervised and traceable**, with an identified approver.

## 15. Platform security

Oktavia has read access and, when the customer authorizes it, change access to their network. Its own security is part of the scope, not an annex.

### Where the data lives

The platform is deployed on the customer infrastructure, on premises or in their private cloud. Inventory, configurations, and artifacts do not leave it. The only exception is AI analysis: when an action that consumes the model runs, the content that query needs travels to the AI provider. Argos states before every action whether it will consume it, so the decision is the operator's, case by case, and the deterministic capabilities cover most of the work without sending anything. The local AI, marked ◆ IA local, runs on the same server and sends nothing either; its index never stores secrets.

### Device credentials

They are stored encrypted at rest in the database, with a key that lives in the deployment configuration and not in the code. They are never shown in the interface nor written to logs. We recommend read-only credentials for discovery, and separate change credentials only if remediation is in scope.

### Access to the platform

Mandatory login, integrated with the corporate identity provider. On top of that, an access allowlist by IP address or network, managed from the interface itself, with two modes: **monitor**, which logs rejections without blocking, and **strict**, which blocks. Switching to strict is guarded: it cannot be enabled if the operator own address is not covered. The same list applies to the web interface and to the automation engine.

### Data separation

Multi-tenant with row-level separation: every query is scoped to the user organization. One customer data is not reachable from another customer session, not even through a programming mistake.

### Between components

The web interface and the automation engine authenticate with a key on every call, in both directions. The engine exposes no operation without it.

### Scoped API keys

An external integration never receives the interface key: it gets its own key, scoped to one organization, read or write, and to a closed list of operations. It is revoked without touching the others.

### No change without approval

This is a security property too, not only a process one: no action that writes to the network happens without a person approving it, and every application is recorded with its approver, its diff, and its artifact.

### Corporate identity and second factor

Login integrates with the customer identity provider over OpenID Connect. Oktavia supports **Okta**, and with it single sign-on and whatever second factor the provider policy already enforces: onboarding, offboarding,

and passwords live where the corporate directory already lives, with no parallel user list to maintain. Local passwords are kept for contingency only.

The platform can also require the authentication to be recent, not merely that a session exists at the provider: it can demand a fresh authentication event, restrict the accepted methods, a passkey for example, and require a specific assurance level, such as the Okta two-factor one. That is what makes closing the browser ask for the second factor again.

> **Confidentiality.** Handling of customer information is governed by the confidentiality agreement signed at the start of the project, with the scope, retention, and return or destruction of the data agreed there.

## 16. Integration requirements

To deploy Oktavia over an existing network we ask the customer for:

- Base device inventory.
- List of sites and site types.
- Operational roles per device.
- Vendor, platform, and version, where the data exists.
- Management addresses and access method.
- Read credentials for discovery.
- Change credentials, if remediation is in scope.
- Backups or available live configuration.
- Current configuration standards.
- Operational constraints and change windows.
- Approvers and technical contacts.

### Minimum data per device

Identity (hostname, management address, site, role), operating system and model, interfaces, layer 2 discovery neighbors, BGP and OSPF neighbors where they exist, live configuration or backup, and the baseline operational services: NTP, DNS, SNMP, SSH, banners, and time zone.

### Preferred order for sourcing the data

1. Native vendor API.
2. NETCONF, RESTCONF, or remote procedure calls.
3. Structured vendor parsers.
4. Command-line output parsing.
5. Live configuration or backups.

## Server for Oktavia

Oktavia runs on a single Linux x86_64 server (Ubuntu 22.04 or 24.04 LTS recommended) with Docker and Docker Compose v2, physical or virtual, on premises or in the customer's cloud. The figures are estimates for sizing the machine: they depend on the number of interfaces per device, how often audits run, and how long telemetry is kept.

**Oktavia Pro** includes inventory and discovery, configuration and state compliance, the Digital Twin with approved remediation, backups, and Argos.

| DEVICES | VCPU | RAM | SSD DISK |
|---|---|---|---|
| Up to 20 (proof of concept) | 4 | 8 GB | 60 GB |
| 20 to 100 | 4 | 16 GB | 100 GB |
| 100 to 300 | 8 | 16 GB | 150 GB |
| 300 to 1,000 | 16 | 32 GB | 250 GB |
| More than 1,000 | Sized with the customer (separate audit workers and database) | | |

**Oktavia Pro with Telemetry and Event Orchestration** adds SNMP collection every 30 seconds, syslog reception, the time-series database with 7 days of retention, and the always-on event evaluator.

| DEVICES | VCPU | RAM | SSD DISK |
|---|---|---|---|
| Up to 20 (proof of concept) | 4 | 8 GB | 80 GB |
| 20 to 100 | 8 | 16 GB | 150 GB |
| 100 to 300 | 8 | 32 GB | 250 GB |
| 300 to 1,000 | 16 | 64 GB | 500 GB |
| More than 1,000 | Sized with the customer (separate time-series database) | | |

> **Proof of concept with 10 or 20 devices:** a single virtual machine with 4 vCPU, 8 GB of RAM, and an 80 GB SSD covers Oktavia Pro and telemetry. With 10 devices there is plenty of headroom. In our lab the full platform uses about 1.5 GB of RAM at rest; the rest is headroom for parallel audits, the operating system, and growth.

## Considerations

- Telemetry disk grows with the interfaces per device and with retention: keeping 30 days instead of 7 multiplies that part by four.
- Backups and audit results grow with audit frequency; the retention policy is agreed during implementation.

- Argos with a cloud AI provider needs no extra resources. The local AI needs at least 16 GB of additional RAM and, preferably, a GPU: on CPU an answer takes roughly a minute.
- Connectivity: the server must reach the devices' management plane (SSH, HTTPS/API, NETCONF, and SNMP depending on the vendor) and receive syslog (UDP 514) if telemetry is enabled. Users reach the console over HTTPS.

## 17. Implementation process

1. **Data onboarding.** Receive the base inventory, normalize names, roles, sites, platforms, and models, and load the initial file.
2. **Controlled discovery.** Validate reachability, run discovery in batches, collect backups, and normalize the observed inventory.
3. **Intent baseline.** Define services per scope, load service variables, enable compliance checks, and configure the customer service catalog.
4. **Validation.** Run the full audit, review the dashboards, and tune rules and exceptions.
5. **Operation and remediation.** Propose the change, simulate it, approve it, apply it, and recompute compliance to confirm closure.

### Estimated time to a working deployment

| NETWORK SIZE | ESTIMATED TIME |
|---|---|
| Small network | 1 to 3 days |
| Medium network | 3 to 7 days |
| Large or complex multi-vendor network | 2 to 4 weeks |

> AI accelerates the preparation: inventory cleanup, role and platform normalization, parser prototyping, and catalog generation. Final validation of intent, compliance, and changes stays operational and human.

## 18. Licensing

The complete platform is **Oktavia Pro**. Continuous telemetry and event orchestration are licensed separately because they have a different consumption profile, they store time series and run a permanent evaluator, and not every network needs them on day one.

### Oktavia Pro

Multi-vendor discovery, inventory, configuration and state compliance, desired configuration per device and per scope, the full remediation cycle with Digital Twin, new service rollout with rollback, topology with path tracing and firewall policy analysis, network security, service inventory, IP Intelligence, configuration backups, advanced reports, provisioning, scheduled operations, FinOps, multi-tenant, and the complete Argos with the local AI.

### Telemetry and Event Orchestration

Real-time metrics, telemetry dashboards compatible with Grafana, and the event automation engine: condition-to-action rules that react automatically to BGP session drops, OSPF neighbor drops, interfaces down, and CPU, memory, temperature, and error thresholds.

### Included with every license: support and updates

While the license is active, every Oktavia deployment receives the platform's security patches, updates and new features, at no extra cost.

## 19. Use cases

- Configuration compliance audit across multiple sites in minutes.
- Automatic detection of configuration drift after a maintenance window.
- Desired configuration generation for new devices by role and by site.
- Live path trace between two points with next hop, egress interface, and asymmetry detection.
- Capacity visibility on wide area interfaces before saturation.
- Post-maintenance connectivity validation across the whole network.
- Change control with a diff between desired and live configuration before applying.
- Onboarding a new network with site types and sites configured before the first discovery run.
- Firewall security posture with the configuration evidence backing each finding.
- Discovering which services are configured and audited by nobody, and declaring them without touching the network.
- Knowing whether a flow passes: which firewall policy decides it at each hop, with its address translation and its logging.
- Locating an IP address down to the access port by crossing the ARP table with the MAC table.
- Adopting the configuration a device already has as declared intent, to start auditing it without redesigning anything.
- Fixing a deviation end to end: propose, approve, apply, and verify it closed.
- Bringing up a tunnel between two sites with its routes, and tearing it down completely when the project ends.
- Advertising a new network in OSPF and BGP on the site edge devices, with ping verification included.
- Asking what the runbook or the vendor guide says during an incident, with the citation, without the documentation leaving the server.

- Preparing the PCI DSS network report for the point-of-sale sites, with evidence per device, for the QSA's review.
- Rolling out a customer's L3VPN or pseudowire on the IOS XR core, with commit confirmed and verification of IS-IS, BGP and the xconnect.
- Following a customer's path across the MPLS L3VPN: ingress PE, VPN label and egress PE, forward and back.
- Turning exploited CVEs with the feature enabled into security patches ready for approval, and verifying the CVE disappeared.

---

## About Oktanet

Oktanet is a Mexican network automation company focused on controlled, traceable, and scalable operations for enterprise environments in Latin America.

For a demonstration on your own environment, reach us at oktanet.io.

**LinkedIn** · **YouTube**

© 2026 Oktanet · Oktavia · Public document, with no confidential customer information.